Ensemble of SVM and Decision Tree Models to Forecast Power Consumption in Indian Farmlands

Cloud Computing Lab - Project Report

Student name	Roll number
Ankit Garg	Btech/10218/23
Ayush Mishra	Btech/10219/23
Anshuman Banerjee	Btech/10211/23

Course: Cloud Computing Lab (B.Tech)
Semester: 7th
Submission date: 3 November 2026

Code repository: https://github.com/AnKiT-GaRG2/farm-power-forecast
Deployed application: https://farm-power-forecast-ankit.streamlit.app/

Abstract

Agriculture, mainly electric irrigation pumpsets, is one of the largest electricity-consuming sectors in India, so accurate short-term demand forecasts help utilities plan generation and avoid shortages during sowing and irrigation seasons. This project builds a forecasting system for the daily electricity consumption of 15 agriculture-dominated Indian states using a Support Vector Regressor (SVR), a Decision Tree Regressor (DT) and two ensembles of the two (soft-voting average and stacking with a ridge meta-learner). Models use lagged consumption, a 7-day rolling mean and calendar features and are evaluated on a chronologically held-out period starting 2020-02-14. The SVR had the lowest MAE (16.88 MU/day) and MAPE, while the voting ensemble had the lowest RMSE (32.97) and highest R2 (0.902) and was better than the single decision tree on MAE, MAPE and RMSE (MAE 17.45 vs 18.95). The stacking ensemble was the weakest model. All differences between SVR, DT and voting are small relative to the error caused by a regime change (COVID-19 lockdown) inside the test period. The trained models are served through a Streamlit dashboard deployed on a cloud platform.

1. Introduction

1.1 Background

India is the world's third-largest electricity producer and consumer. Agricultural consumption is dominated by groundwater pumping, which is strongly seasonal and depends on rainfall, crop cycle and state farm-power policy. Forecasting demand ahead of time is a core task in grid operation.

1.2 Motivation

Single machine-learning models each have weaknesses: SVR is smooth but sensitive to hyper-parameters and scaling, while a decision tree captures non-linear rules but gives piecewise-constant, high-variance output. Combining heterogeneous learners is a standard way to reduce variance, and the assignment asks to compare the individual models with their ensemble.

1.3 Problem being addressed

Predict the next-day power consumption (million units, MU) of agrarian states using historical consumption and calendar information, and quantify whether an SVM + decision-tree ensemble improves over each individual model.

2. Problem Statement and Objectives

Develop an ensemble of SVM and decision tree models to forecast power consumption in Indian farmlands, using a suitable Kaggle dataset, deploy the model on a cloud platform, and compare individual performance with the ensemble.

- Select and document a suitable public Kaggle dataset and justify its use for agrarian-region forecasting.

- Engineer time-series features and train SVR and Decision Tree regressors with time-aware hyper-parameter tuning.
- Build voting and stacking ensembles of the two models.
- Compare all models with MAE, RMSE, R2, MAPE, training time and a naive baseline.
- Build an interactive dashboard and deploy it on a cloud platform.

3. System Architecture and Methodology

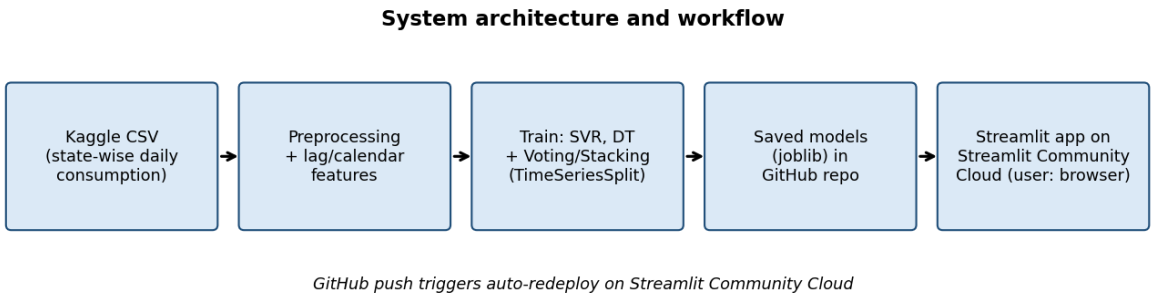


Figure 1: Architecture and workflow

3.1 Workflow

- Load the long-format CSV, convert dates (dd/mm/yyyy), rename UP/MP to full state names, keep 15 agrarian states.
- Create per-state features: lag-1, lag-2, lag-7, previous-7-day rolling mean, day-of-week, month, sine/cosine of day-of-year, and the state (one-hot).
- Chronological 80/20 split by date; hyper-parameters tuned on the training part only with 3-fold TimeSeriesSplit.
- Train SVR, Decision Tree, Voting and Stacking ensembles; evaluate on the held-out period; persist with joblib.
- Commit models and app to GitHub; Streamlit Community Cloud builds and serves the dashboard.

3.2 Algorithms and formulation

Support Vector Regression (RBF kernel) fits f(x) = sum_i (a_i - a_i*) K(x_i, x) + b with epsilon-insensitive loss, K(x, x') = exp(-gamma ||x - x'||^2). Hyper-parameters C, gamma and epsilon were tuned by grid search.

Decision Tree Regression recursively splits the feature space to minimise the squared error within leaves; max_depth and min_samples_leaf were tuned.

Voting ensemble: y_hat = (y_svr + y_dt) / 2. Stacking ensemble: y_hat = RidgeCV(y_svr, y_dt) where the meta-learner is trained on 5-fold out-of-fold predictions of the base models.

Metrics: MAE = mean|y - y_hat|; RMSE = sqrt(mean (y - y_hat)^2); R2 = 1 - SSE/SST; MAPE = 100 * mean|(y - y_hat)/y|.

4. Dataset and Experimental Configuration

Item	Value
Dataset name	Power consumption in India (2019-2020)
Source	Kaggle, author: twinkle0705 (Twinkle Khanna); licence CC0 Public Domain
URL	https://www.kaggle.com/datasets/twinkle0705/state-wise-power-consumption-in-india
Instances	16,599 rows (33 states x 503 days, 2 Jan 2019 - 23 May 2020; five 1-day gaps: 1 Feb-1 Jun 2019); 7,440 rows used (15 states x 496 days after lagging)
Features	Raw: States, Regions, latitude, longitude, Dates, Usage (6). Model inputs: 8 numeric + 1 categorical
Target variable	Usage - daily electricity consumption in million units (MU)
Preprocessing	No missing values; repair of day/month-swapped dates (see below); lag/rolling/calendar features; StandardScaler on numeric, one-hot on state
Train/test methodology	Chronological split: train < 2020-02-14 (5940 rows), test >= 2020-02-14 (1500 rows)

Data quality issue and repair: the date column of the Kaggle CSV contains a day/month swap (an Excel locale artefact). 35 rows (true dates 6-12 January-May 2020) are stored as mm/dd and would parse as June-December 2020, and the 7th of August-December 2019 is stored as mm/dd and collides with 8-12 July 2019. We re-parse dates after 23 May 2020 (the end of the data) as mm/dd and resolve the five colliding pairs by continuity of the all-state total with the previous day. The pair assignment is a heuristic; its effect is confined to 10 of 7,440 modelled rows (about 0.1%).

Assumption and limitation: the dataset reports total state consumption, not farm-only consumption. Agriculture-dominated states were chosen as a proxy, because their load is strongly influenced by irrigation pumping; the dataset does not separate agricultural load. This is documented as a limitation.

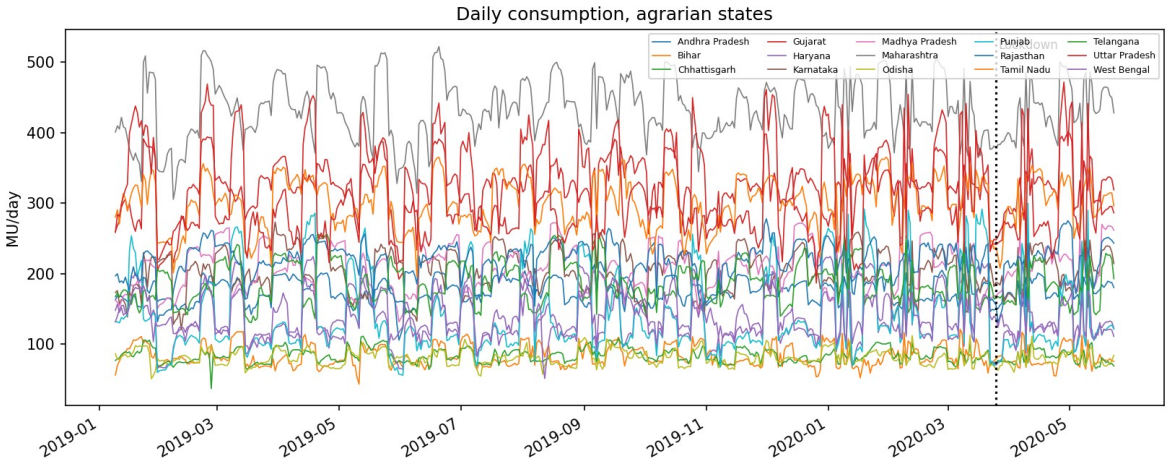


Figure 2: Daily consumption of the selected states (dotted line: nationwide lockdown, 25 Mar 2020)

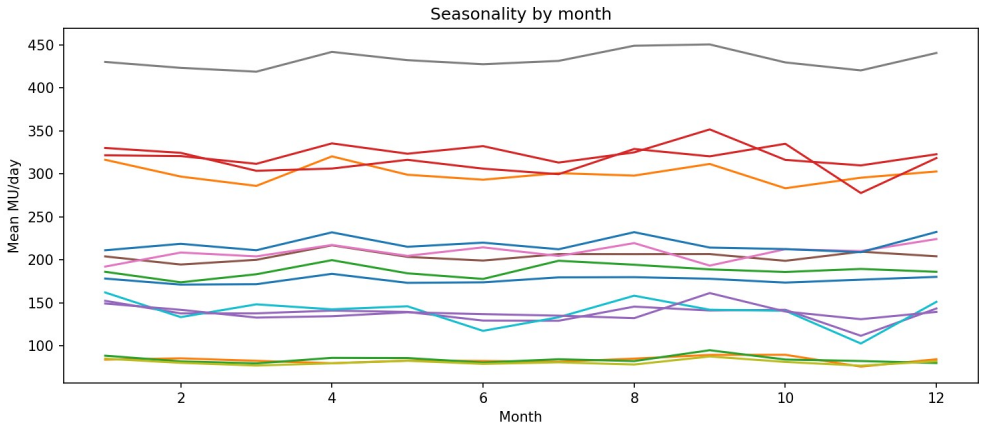


Figure 3: Monthly mean consumption showing seasonality

5. Implementation

- Language/libraries: Python 3.10, pandas, NumPy, scikit-learn 1.7.2 (SVR, DecisionTreeRegressor, VotingRegressor, StackingRegressor, GridSearchCV), matplotlib, Plotly.
- Dashboard: Streamlit with three tabs - model comparison, forecast explorer (predicted vs actual per state) and what-if prediction.
- Cloud platform: Streamlit Community Cloud (free tier), source from GitHub; Python environment from requirements.txt.
- Repository layout: src/train.py (training), src/build_report.py, app.py (dashboard), models/ (trained artefacts), figures/, data/.

Best SVR parameters: {'m__C': 300, 'm__epsilon': 0.1, 'm__gamma': 0.01}. Best Decision Tree parameters: {'m__max_depth': 8, 'm__min_samples_leaf': 20}.

6. Experimental Setup

- Hardware/software: personal laptop, Linux, Python 3.10, scikit-learn 1.7.2.
- Dataset/workload: as in Section 4.
- Parameters: SVR grid C in {10,100,300}, gamma in {scale,0.01}, epsilon in {0.1,1.0}; DT grid max_depth in {4,6,8,12,None}, min_samples_leaf in {1,5,10,20}; random_state=42.
- Number of runs: one chronological train/test split (deterministic); the tuning used 3-fold TimeSeriesSplit.
- Metrics: MAE, RMSE, R2, MAPE, training time; naive baseline (yesterday's value).

7. Results

7.1 Overall comparison

Model	MAE	RMSE	R2	MAPE_%	train_time_s
SVR	16.876	33.751	0.897	9.265	21.490
Decision Tree	18.946	33.168	0.901	10.496	0.710
Ensemble (Voting)	17.454	32.968	0.902	9.636	1.290
Ensemble (Stacking)	19.574	35.999	0.883	10.802	5.970
Naive (lag1)	17.299	34.549	0.892	9.519	0.000

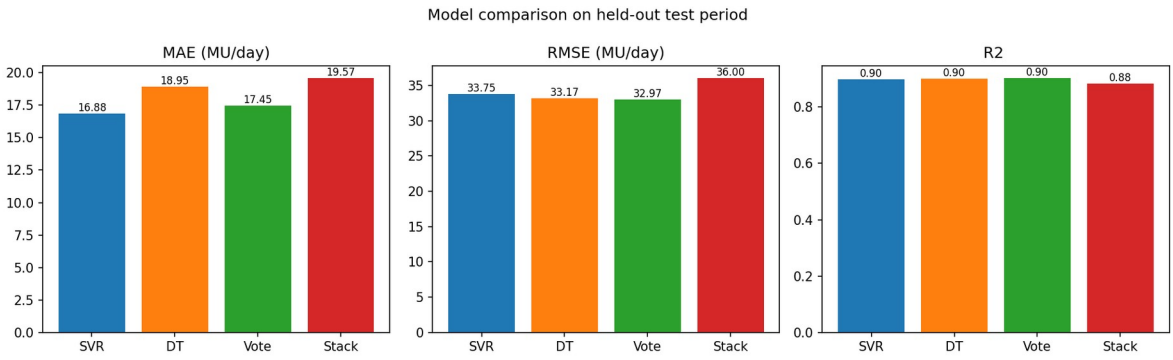


Figure 4: MAE, RMSE and R2 per model

7.2 Predicted vs actual

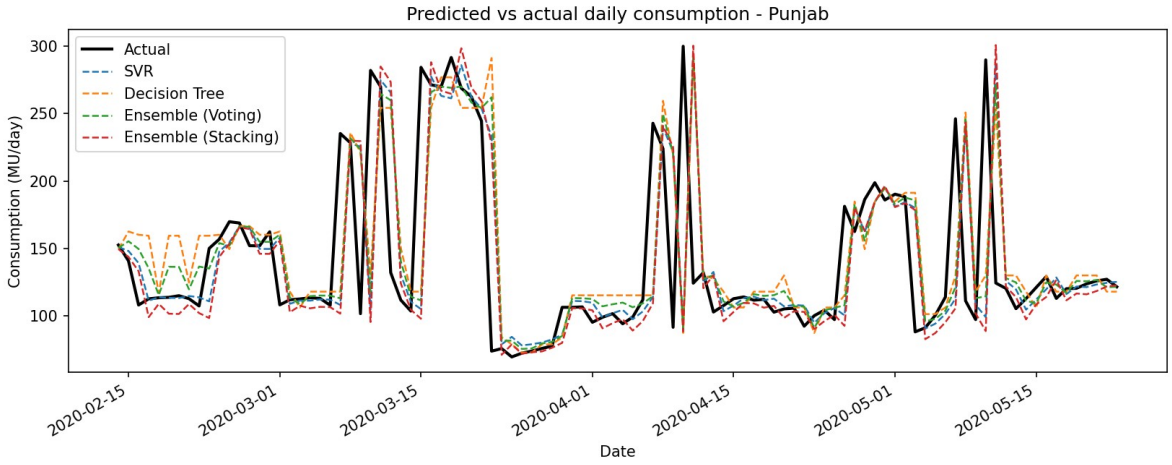


Figure 5: Predicted vs actual daily consumption for Punjab (test period)

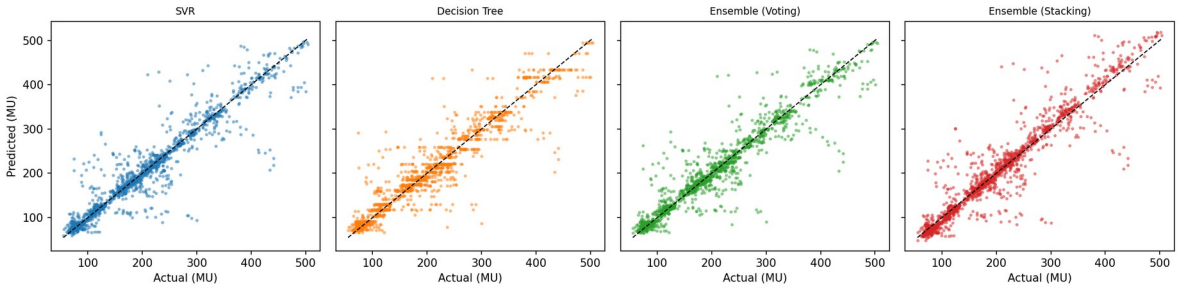


Figure 6: Predicted vs actual scatter plots (dashed line = perfect prediction)

7.3 Errors

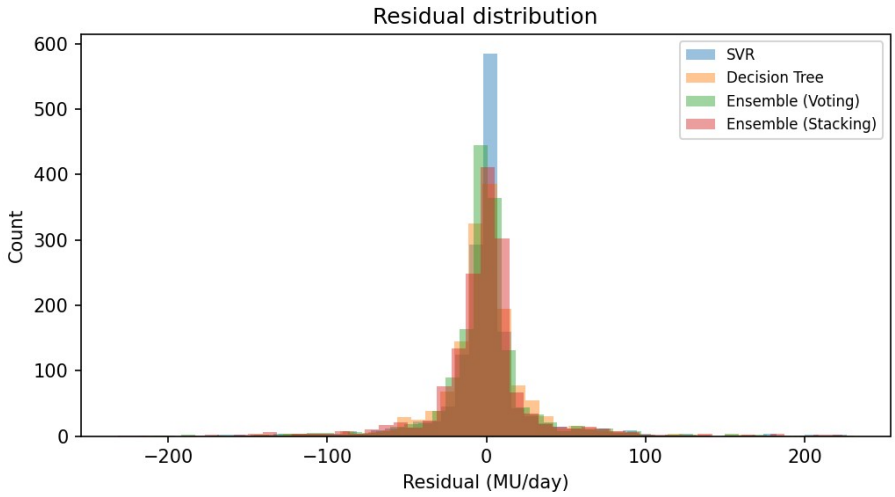


Figure 7: Residual distributions

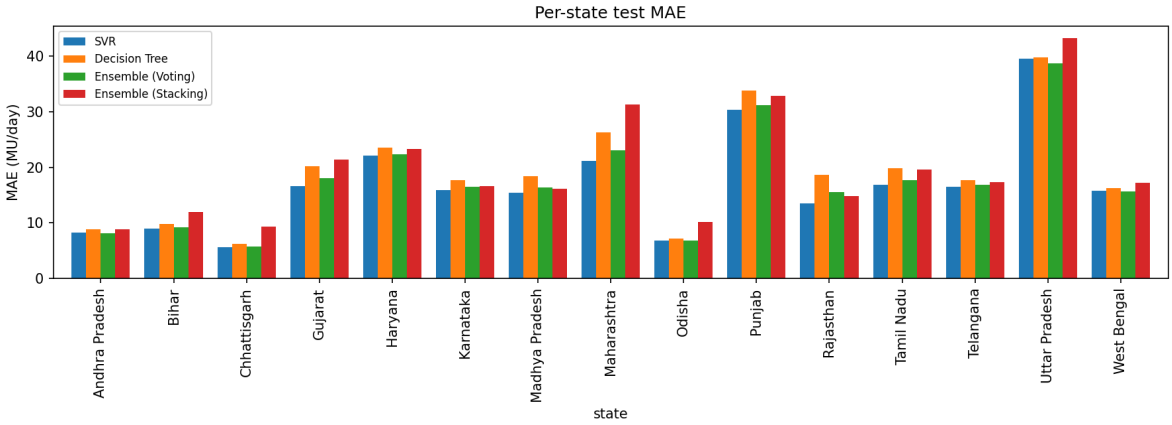


Figure 8: MAE per state

State	SVR	Decision Tree	Ensemble (Voting)	Ensemble (Stacking)
Andhra Pradesh	8.17	8.85	8.15	8.76
Bihar	8.88	9.74	9.16	11.90
Chhattisgarh	5.53	6.17	5.70	9.27
Gujarat	16.63	20.23	18.01	21.33
Haryana	22.12	23.58	22.39	23.24
Karnataka	15.94	17.72	16.50	16.56
Madhya Pradesh	15.34	18.34	16.40	16.10
Maharashtra	21.17	26.30	23.11	31.36
Odisha	6.76	7.15	6.78	10.08
Punjab	30.36	33.78	31.18	32.85
Rajasthan	13.53	18.64	15.57	14.81
Tamil Nadu	16.89	19.89	17.71	19.56
Telangana	16.50	17.70	16.86	17.31
Uttar Pradesh	39.54	39.84	38.68	43.27
West Bengal	15.77	16.29	15.62	17.21

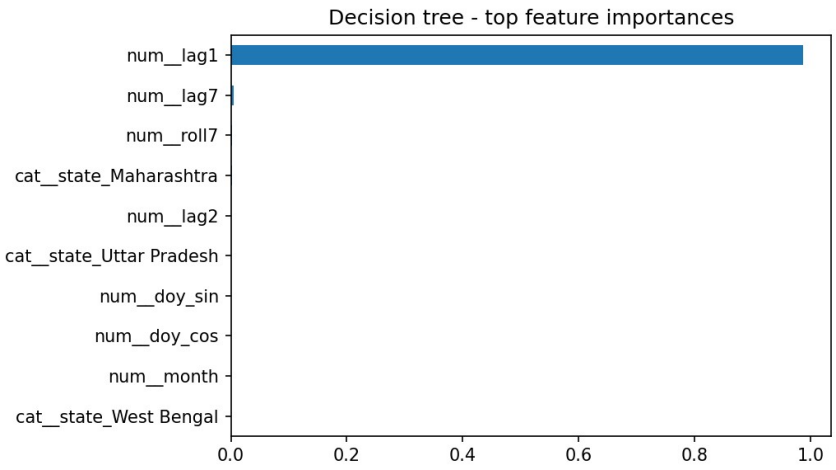


Figure 9: Decision-tree feature importances

7.4 Deployment

- Cloud platform: Streamlit Community Cloud.
- Architecture: GitHub repo -> Streamlit build (pip install requirements.txt) -> app served over HTTPS.
- Services/resources: Streamlit Community Cloud container (shared CPU, ~1 GB RAM), GitHub repository.
- Procedure: push the repo to GitHub; on share.streamlit.io choose New app, select repository, branch main, main file app.py; click Deploy.
- Application URL: https://farm-power-forecast-ankit.streamlit.app/
- Screenshots: see Appendix A (the running application on Streamlit Community Cloud).
- Challenges: pinning scikit-learn to the version used for training (pickled models are version-sensitive); keeping model files small enough for the free tier; cold-start delay when the app has been idle.

8. Analysis and Discussion

What worked: all learners reach R2 of about 0.88-0.90 because yesterday's consumption is a strong predictor. SVR has the lowest MAE (16.88) and MAPE (9.3%). The voting ensemble has the best RMSE (32.97 vs 33.75 for SVR and 33.17 for DT) and best R2 (0.902), i.e. it makes the fewest large errors, and it lowers the decision tree's MAE from 18.95 to 17.45 (8% lower) and MAPE from 10.5% to 9.6%. This is the expected variance-reduction effect of averaging two different learners.

What did not: the voting ensemble is not better than SVR on MAE (17.45 vs 16.88), and its RMSE advantage over SVR (about 2%) and R2 advantage (0.005) are small; with one split and no confidence intervals we cannot claim it is statistically significant. Stacking was the worst model (MAE 19.57, RMSE 36.00), worse even than its two base learners. A likely reason (not tested) is that its meta-learner was fitted on 5-fold out-of-fold predictions that are not time-ordered, so its weights do not transfer to the shifted test period. The naive 'yesterday' baseline

(MAE 17.30, R2 0.892) is about as good as the learned models: it beats the decision tree and stacking on MAE and is within 3% of SVR, so the learned models add only modest value over persistence on this dataset.

Why: the test period (from 14 Feb 2020 to 23 May 2020) contains the national COVID-19 lockdown starting 25 March 2020, when consumption changed abruptly. Models trained on 2019 and early-2020 data have never seen such a regime, so errors are dominated by that structural break (visible in Figure 5 and the long residual tails in Figure 7). Punjab and Haryana also show sharp step changes (possibly linked to agricultural supply schedules; we did not verify the cause), which a one-day-ahead model can only follow after the change has happened.

Trends: states with large and volatile load (Uttar Pradesh, Punjab, Haryana, Maharashtra) have the largest absolute errors; smaller consumers (Chhattisgarh, Odisha, Bihar, Andhra Pradesh) have the smallest. The decision tree's feature importance is dominated by lag-1 consumption (Figure 9).

Limitations: (1) only 17 months of data, so a full seasonal cycle is never repeated and the test set is atypical; (2) state totals rather than farm-level data; (3) no weather, rainfall, crop or tariff covariates; (4) a single split and no confidence intervals, so differences of a few percent between models should not be over-interpreted; (5) date repair heuristic (Section 4); (6) five missing days in 2019 make lag-1/lag-7 features span two days at those points; (7) the ensemble result is specific to this dataset and period.

9. Conclusion

An SVR, a decision tree and two SVR+DT ensembles were built and compared for forecasting daily power consumption of 15 agrarian Indian states. SVR achieved the lowest MAE (16.88 MU/day) and MAPE; the voting ensemble achieved the lowest RMSE (32.97) and highest R2 (0.902) and improved clearly on the decision tree, but the margin over SVR is small. Stacking did not help. The work was deployed as an interactive Streamlit dashboard on a cloud platform. The main findings are that ensembling gives a modest, not dramatic, gain over the best individual model here, and that regime changes such as the lockdown, together with data-quality issues in the public dataset, limit all models.

10. Future Scope

- Use farm-level or feeder-level agricultural load data (for example state discom pumpset consumption) and multi-year data.
- Add weather, rainfall, groundwater and crop-calendar features.
- Weighted ensembles or gradient boosting / LSTM comparison; walk-forward evaluation.
- Multi-day-ahead forecasting with prediction intervals; automated retraining pipeline on the cloud.

11. References

- Khanna, T. (2020). Power consumption in India (2019-2020). Kaggle. https://www.kaggle.com/datasets/twinkle0705/state-wise-power-consumption-in-india (CC0).
- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825-2830.
- Drucker et al. (1997). Support Vector Regression Machines. NIPS 9.
- Breiman et al. (1984). Classification and Regression Trees. Wadsworth.
- Wolpert, D. (1992). Stacked Generalization. Neural Networks 5(2).
- Streamlit documentation: https://docs.streamlit.io; Streamlit Community Cloud: https://streamlit.io/cloud
- pandas, NumPy, matplotlib and Plotly documentation.

Appendix A: Deployment screenshots

The application was deployed at https://farm-power-forecast-ankit.streamlit.app/ on Streamlit Community Cloud from the GitHub repository (branch main, main file app.py).

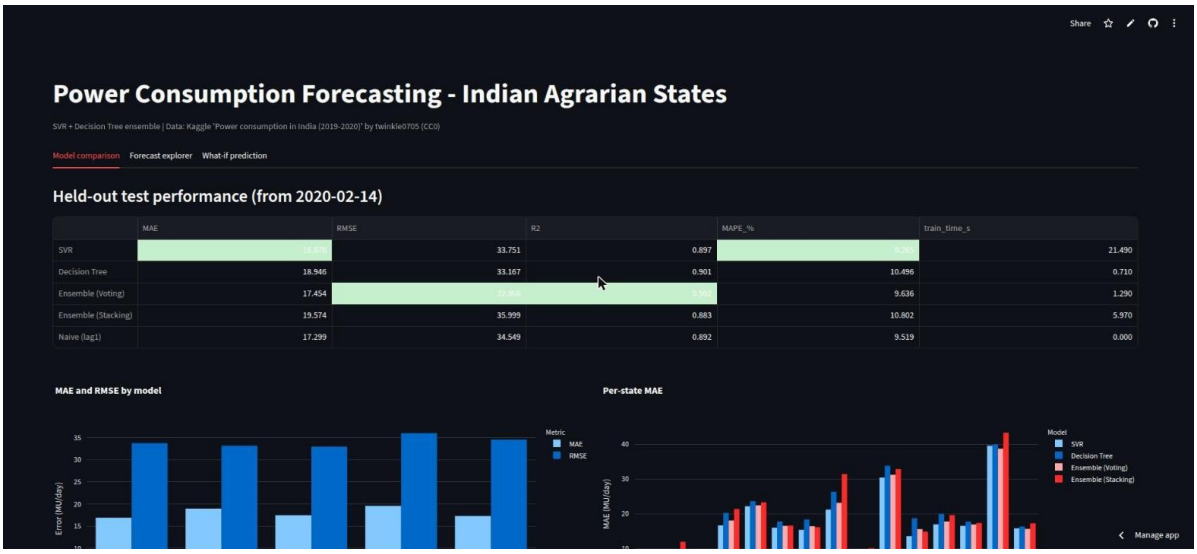


Figure A1: Deployed dashboard - model comparison tab

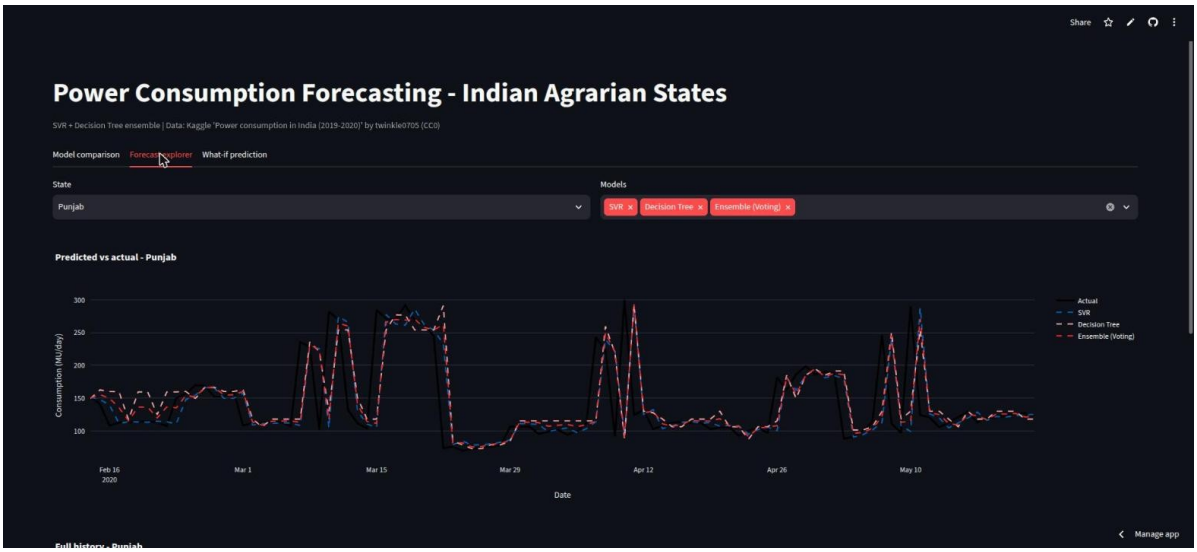


Figure A2: Deployed dashboard - forecast explorer tab (Punjab)